Experiment 7

AIM:

Given a denormalized Sales table, identify functional dependencies and normalize step-by-step from UNF to 3NF and BCNF. Verify lossless join and dependency preservation using SQL queries.

SOURCE CODE:

1. Create Database and Sales Table (UNF):

```
CREATE DATABASE
SalesNormalization; USE
SalesNormalization;
CREATE TABLE Sales_UNF (
  Order_ID INT,
  Order_Date DATE,
  Customer_ID VARCHAR(10),
  Customer_Name
  VARCHAR(50), Product_ID
  VARCHAR(10), Product_Name
  VARCHAR(50), Quantity INT,
  Unit_Price DECIMAL(10,2),
  Salesperson_ID VARCHAR(10),
  Salesperson_Name VARCHAR(50)
);

INSERT INTO Sales_UNF VALUES
(1001, '2026-01-10', 'C01', 'Aarav', 'P01', 'Laptop', 2,
50000, 'S01', 'Rahul'),
(1001, '2026-01-10', 'C01', 'Aarav', 'P02', 'Mouse', 3, 500, 'S01', 'Rahul'),
(1002, '2026-01-12', 'C02', 'Priya', 'P01', 'Laptop', 1,
50000, 'S02', 'Neha'),
(1003, '2026-01-15', 'C03', 'Rohan', 'P03', 'Keyboard', 2,
1500, 'S01', 'Rahul'),
(1004, '2026-01-18', 'C01', 'Aarav', 'P03', 'Keyboard', 1,
1500, 'S02', 'Neha'),
(1005, '2026-01-20', 'C04', 'Meera', 'P02', 'Mouse', 4, 500, 'S03', 'Amit');

SELECT * FROM Sales_UNF;
```

Order_ID	Order_Date	Customer_ID	Customer_Name	Product_ID	Product_Name	Quantity	Unit_Price	Salesperson_ID	Salesperson_Name
1001	2026-01-10T00:00:00.000Z	C01	Aarav	P01	Laptop	2	50000.00	S01	Rahul
1001	2026-01-10T00:00:00.000Z	C01	Aarav	P02	Mouse	3	500.00	S01	Rahul
1002	2026-01-12T00:00:00.000Z	C02	Priya	P01	Laptop	1	50000.00	S02	Neha
1003	2026-01-15T00:00:00.000Z	C03	Rohan	P03	Keyboard	2	1500.00	S01	Rahul
1004	2026-01-18T00:00:00.000Z	C01	Aarav	P03	Keyboard	1	1500.00	S02	Neha
1005	2026-01-20T00:00:00.000Z	C04	Meera	P02	Mouse	4	500.00	S03	Amit

2. Insert Sales Records:

```
INSERT INTO Sales_1NF
SELECT * FROM
Sales_UNF; SELECT *
FROM Sales_1NF;
```

Order_ID	Order_Date	Customer_ID	Customer_Name	Product_ID	Product_Name	Quantity	Unit_Price	Salesperson_ID	Salesperson_Name
1001	2026-01-10T00:00:00.000Z	C01	Aarav	P01	Laptop	2	50000.00	S01	Rahul
1001	2026-01-10T00:00:00.000Z	C01	Aarav	P02	Mouse	3	500.00	S01	Rahul
1002	2026-01-12T00:00:00.000Z	C02	Priya	P01	Laptop	1	50000.00	S02	Neha
1003	2026-01-15T00:00:00.000Z	C03	Rohan	P03	Keyboard	2	1500.00	S01	Rahul
1004	2026-01-18T00:00:00.000Z	C01	Aarav	P03	Keyboard	1	1500.00	S02	Neha
1005	2026-01-20T00:00:00.000Z	C04	Meera	P02	Mouse	4	500.00	S03	Amit

3. Create Sales Table in 1NF:

```
CREATE TABLE Orders (
  Order_ID INT PRIMARY KEY,
  Order_Date DATE, Customer_ID
  VARCHAR(10),
  Customer_Name
  VARCHAR(50), Salesperson_ID
  VARCHAR(10),
  Salesperson_Name
  VARCHAR(50)
);
INSERT INTO Orders
SELECT DISTINCT
      Order_ID, Order_Date,
      Customer_ID,
      Customer_Name,
      Salesperson_ID,
      Salesperson_Name
FROM Sales_1NF;
CREATE TABLE Products (
  Product_ID VARCHAR(10) PRIMARY KEY,
  Product_Name VARCHAR(50),
  Unit_Price DECIMAL(10,2)
);
INSERT INTO Products
SELECT DISTINCT
  Product_ID, Product_Name,
```

```
  Unit_Price
FROM Sales_1NF;
CREATE TABLE Order_Details (
  Order_ID INT,
  Product_ID VARCHAR(10),
  Quantity INT,
  PRIMARY KEY (Order_ID, Product_ID),
  FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID),
  FOREIGN KEY (Product_ID) REFERENCES Products(Product_ID)
);

INSERT INTO Order_Details
SELECT Order_ID, Product_ID, Quantity FROM
Sales_1NF;
```

4. Create Orders Table:

```
SELECT * FROM Orders;
```

Order_ID	Order_Date	Customer_ID	Customer_Name	Salesperson_ID	Salesperson_Name
1001	2026-01-10T00:00:00.000Z	C01	Aarav	S01	Rahul
1002	2026-01-12T00:00:00.000Z	C02	Priya	S02	Neha
1003	2026-01-15T00:00:00.000Z	C03	Rohan	S01	Rahul
1004	2026-01-18T00:00:00.000Z	C01	Aarav	S02	Neha
1005	2026-01-20T00:00:00.000Z	C04	Meera	S03	Amit

5. Create Products Table:

```
SELECT * FROM Products;
```

Product_ID	Product_Name	Unit_Price
P01	Laptop	50000.00
P02	Mouse	500.00
P03	Keyboard	1500.00

6. Create Order Details Table:

```
SELECT * FROM Order_Details;
```

Order_ID	Product_ID	Quantity
1001	P01	2
1001	P02	3
1002	P01	1
1003	P03	2
1004	P03	1
1005	P02	4

7. Create Customers Table:

```
CREATE TABLE Customers (
  Customer_ID VARCHAR(10) PRIMARY KEY,
  Customer_Name  VARCHAR(50)
);
INSERT INTO Customers
SELECT DISTINCT Customer_ID, Customer_Name
FROM Sales_1NF;
CREATE TABLE Salespersons (
  Salesperson_ID VARCHAR(10) PRIMARY
  KEY, Salesperson_Name VARCHAR(50)
);
INSERT INTO Salespersons
SELECT DISTINCT Salesperson_ID, Salesperson_Name
FROM Sales_1NF;
CREATE TABLE Orders_3NF (
  Order_ID INT PRIMARY KEY,
Order_Date DATE,

  Customer_ID VARCHAR(10),
  Salesperson_ID VARCHAR(10),
  FOREIGN KEY (Customer_ID)
  REFERENCES Customers(Customer_ID),
  FOREIGN KEY (Salesperson_ID)
  REFERENCES Salespersons(Salesperson_ID)
);
INSERT INTO Orders_3NF
SELECT DISTINCT
  Order_ID, Order_Date,
  Customer_ID,
  Salesperson_ID
FROM Sales_1NF;
SELECT * FROM Customers;
```

Customer_ID	Customer_Name
C01	Aarav
C02	Priya
C03	Rohan
C04	Meera

8. Create Salespersons Table:

SELECT * FROM Salespersons;

Salesperson_ID	Salesperson_Name
S01	Rahul
S02	Neha
S03	Amit

9. Create Orders Table in 3NF:

SELECT * FROM Orders_3NF;

Order_ID	Order_Date	Customer_ID	Salesperson_ID
1001	2026-01-10T00:00:00.000Z	C01	S01
1002	2026-01-12T00:00:00.000Z	C02	S02
1003	2026-01-15T00:00:00.000Z	C03	S01
1004	2026-01-18T00:00:00.000Z	C01	S02
1005	2026-01-20T00:00:00.000Z	C04	S03

10. Create Employee Assignment Table for BCNF:

```
CREATE TABLE Employee_Assignment (
  Employee_ID VARCHAR(10),
  Project_ID VARCHAR(10),
  Supervisor_ID VARCHAR(10),
  PRIMARY KEY (Employee_ID, Project_ID)
);
INSERT INTO Employee_Assignment
VALUES ('E01', 'P01', 'S01'),
('E02', 'P01', 'S01'),
('E03', 'P02', 'S02'),
('E04', 'P02', 'S02');
```

11. Create Supervisor Project Table:

```
CREATE TABLE Supervisor_Project (
  Supervisor_ID VARCHAR(10) PRIMARY
  KEY,
  Project_ID VARCHAR(10)
);
INSERT INTO Supervisor_Project VALUES
('S01', 'P01'),
('S02', 'P02');
```

12. Create Employee Supervisor Table:

```
CREATE TABLE Employee_Supervisor (
  Employee_ID VARCHAR(10),
  Supervisor_ID VARCHAR(10),
  PRIMARY KEY (Employee_ID, Supervisor_ID)
);
```

```
INSERT INTO Employee_Supervisor
VALUES ('E01', 'S01'),
('E02', 'S01'),
('E03', 'S02'),
('E04', 'S02');
```

13. Verify BCNF Decomposition:

```
SELECT
  es.Employee_ID,
  sp.Project_ID,
  es.Supervisor_ID
FROM Employee_Supervisor es
JOIN Supervisor_Project sp
  ON es.Supervisor_ID = sp.Supervisor_ID;
```

Employee_ID	Project_ID	Supervisor_ID
E01	P01	S01
E02	P01	S01
E03	P02	S02
E04	P02	S02

14. Verify Lossless Join:

```
SELECT
  o.Order_ID, o.Order_Date,
  c.Customer_ID,
  c.Customer_Name,
  od.Product_ID,
  p.Product_Name,
  od.Quantity, p.Unit_Price,
  s.Salesperson_ID,
  s.Salesperson_Name
FROM Orders_3NF o
JOIN Customers c
  ON o.Customer_ID = c.Customer_ID
JOIN Salespersons s
  ON o.Salesperson_ID = s.Salesperson_ID JOIN
Order_Details od
  ON o.Order_ID = od.Order_ID
JOIN Products p
  ON od.Product_ID = p.Product_ID
ORDER BY o.Order_ID, od.Product_ID;
```

Order_ID	Order_Date	Customer_ID	Customer_Name	Product_ID	Product_Name	Quantity	Unit_Price	Salesperson_ID	Salesperson_Name
1001	2026-01-10T00:00:00.000Z	C01	Aarav	P01	Laptop	2	50000.00	S01	Rahul
1001	2026-01-10T00:00:00.000Z	C01	Aarav	P02	Mouse	3	500.00	S01	Rahul
1002	2026-01-12T00:00:00.000Z	C02	Priya	P01	Laptop	1	50000.00	S02	Neha
1003	2026-01-15T00:00:00.000Z	C03	Rohan	P03	Keyboard	2	1500.00	S01	Rahul
1004	2026-01-18T00:00:00.000Z	C01	Aarav	P03	Keyboard	1	1500.00	S02	Neha
1005	2026-01-20T00:00:00.000Z	C04	Meera	P02	Mouse	4	500.00	S03	Amit

15. Verify Reconstructed Rows:

```
SELECT COUNT(*) AS reconstructed_rows
FROM Orders_3NF o
JOIN Customers c
  ON o.Customer_ID = c.Customer_ID
JOIN Salespersons s
  ON o.Salesperson_ID = s.Salesperson_ID JOIN
Order_Details od
  ON o.Order_ID = od.Order_ID
JOIN Products p
  ON od.Product_ID = p.Product_ID;
```

reconstructed_rows
6

16. Verify Table Constraints:

SHOW CREATE TABLE Customers;

Table	Create Table
Customers	CREATE TABLE `customers` (`Customer_ID` varchar(10) NOT NULL, `Customer_Name` varchar(50) DEFAULT NULL, PRIMARY KEY (`Customer_ID`)) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_09(

SHOW CREATE TABLE Salespersons;

Table	Create Table
Salespersons	CREATE TABLE `salespersons` (`Salesperson_ID` varchar(10) NOT NULL, `Salesperson_Name` varchar(50) DEFAULT NULL, PRIMARY KEY (`Salesperson_ID`)) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=u

SHOW CREATE TABLE Orders_3NF;

Table	Create Table
Orders_3NF	CREATE TABLE `orders_3nf` (`Order_ID` int NOT NULL, `Order_Date` date DEFAULT NULL, `Customer_ID` varchar(10) DEFAULT NULL, `Salesperson_ID` varchar(10) DEFAULT NULL, PRIMARY KEY (`Order_ID`), KEY `Custc

SHOW CREATE TABLE Products;

Table	Create Table
Products	CREATE TABLE `products` (`Product_ID` varchar(10) NOT NULL, `Product_Name` varchar(50) DEFAULT NULL, `Unit_Price` decimal(10,2) DEFAULT NULL, PRIMARY KEY (`Product_ID`)) ENGINE=InnoDB DEFAULT CHARSET=

SHOW CREATE TABLE Order_Details;

Table	Create Table
Order_Details	CREATE TABLE `order_details` (`Order_ID` int NOT NULL, `Product_ID` varchar(10) NOT NULL, `Quantity` int DEFAULT NULL, PRIMARY KEY (`Order_ID`,`Product_ID`), KEY `Product_ID` (`Product_ID`), CONSTRAINT `or

RESULT:

The Sales table was successfully normalized from UNF to 1NF, 2NF, and 3NF by dividing the data into related tables such as Customers, Salespersons, Orders, Products, and Order_Details. Functional dependencies were identified, and redundancy was reduced. The lossless join was verified by reconstructing the original Sales records, and the reconstructed row count was found to be 6. BCNF decomposition was demonstrated using Employee_Assignment, Employee_Supervisor, and Supervisor_Project tables.